



Stochastic Unit Commitment: Model Reduction via Learning

Xuan Liu¹ · Antonio J. Conejo^{1,2} · Gonzalo E. Constante Flores³

Accepted: 15 May 2023 / Published online: 4 July 2023
 © The Author(s), under exclusive licence to Springer Nature Switzerland AG 2023

Abstract

Purpose of Review As weather-dependent renewable generation increases its share in the generation mix of most electric energy systems, a stochastic unit commitment becomes the natural day-ahead scheduling tool. However, such a tool is generally computationally intractable if a detailed uncertainty description is considered. Taking this into account, we proposed a learning method to make the stochastic unit commitment problem tractable.

Recent Findings Recent advances in statistical learning and machine learning to address optimization problems can be advantageously applied to the rather intractable stochastic unit commitment problem. Considering these advances, we explore simple learning techniques to drastically reduce the size of a stochastic unit commitment problem without significantly altering its optimal solution.

Summary The considered stochastic unit commitment problem is formulated as a two-stage stochastic programming problem. The first stage represents commitment decisions, while the second one represents the operation conditions under different scenarios. Taking into account historical solved instances (or proxies for them), we reduce the size (measured by numbers of constraints and variables) of the stochastic unit commitment problem by (i) fixing unchanged binary variables and by (ii) eliminating inactive inequality constraints. Our numerical results show that the reduced problem generally requires significantly less time to solve while obtaining high-quality solutions, which are very close to or indistinguishable from the one obtained by solving the original problem. We use an Illinois 200-bus system to illustrate and characterize the performance of the proposed problem-reduction method.

Keywords Stochastic unit commitment · Learning · Stochastic programming · Mixed-integer linear programming

Notation

The notation used in this paper is provided below for easy reference.

✉ Antonio J. Conejo
 conejo.1@osu.edu

Xuan Liu
 liu.7609@osu.edu

Gonzalo E. Constante Flores
 geconsta@purdue.edu

¹ Department of Electrical and Computer Engineering, The Ohio State University, Columbus 43210, OH, USA

² Department of Integrated Systems Engineering, The Ohio State University, Columbus 43210, OH, USA

³ Davidson School of Chemical Engineering, Purdue University, West Lafayette 47907, IN, USA

A. Indices and Sets

i	Index for weather-dependent generating units in set I .
j	Index for thermal generating units in set J .
ℓ	Index for nuclear generating units in set L .
t	Index for time periods (hours) in set $T = \{1, 2, \dots, T \}$.
ω	Index for scenarios in set Ω .
Ξ	Optimization variables, $\Xi = \Xi^{1\text{stage}} \cup \Xi^{2\text{stage}}$: $\Xi^{1\text{stage}} = \{v_{jt}, y_{jt}, z_{jt}\}$, $\Xi^{2\text{stage}} = \{p_{jt\omega}, p_{it\omega}^{\text{WS}}, D_{t\omega}^{\text{LS}}\}$.

B. Parameters

v_j^{ini}	Commitment status of unit j at the beginning of the planning horizon (1 if online, and 0 otherwise).
p_j^{ini}	Power output of unit j at the beginning of the planning horizon (MW).

$H_j^{D/U}$	Number of hours that unit j has been off/on at the beginning of the planning horizon (h).
$T_j^{D/U}$	Minimum number of hours required for unit j to be off/on before it can be turned on/off (h).
F_j	Constant equal to $T_j^D - H_j^D$ unless it exceeds the number of hours of the planning horizon, in which case F_j equals $ T $, $F_j = \min(T , [T_j^D - H_j^D][1 - v_j^{\text{ini}}])$.
L_j	Constant equal to $T_j^U - H_j^U$ unless it exceeds the number of hours of the planning horizon, in which case L_j equals $ T $, $L_j = \min(T , [T_j^U - H_j^U]v_j^{\text{ini}})$.
$p_{\ell t}^N$	Power output of nuclear unit ℓ in hour t (MW).
$\overline{P}_j/P_j$	Maximum/Minimum power output of unit j (MW).
$R_j^{D/U}$	Maximum ramp-down/up rate of unit j (MW/h).
$S_j^{D/U}$	Maximum shutdown/startup rate of unit j (MW).
$c_j^{U/D}$	Starting up/shutting down cost of unit j (\$).
c_j^{NL}	No load cost of unit j (\$).
c_j^V	Variable cost of unit j (\$/MWh).
c_i^W	Production cost of weather-dependent generating unit i (\$/MWh).
c^{LS}	Load-shedding cost (\$/MWh).
π_ω	Probability of occurrence of scenario ω .
$D_{t\omega}$	Power demand in hour t under scenario ω (MW).
$p_{it\omega}^W$	Power production realization of weather-dependent unit i in hour t under scenario ω (MW).

C. Variables

v_{jt}	Commitment status of unit j in hour t : v_{jt} equals 1 if unit j is on in hour t , and 0 otherwise.
y_{jt}	Startup status of unit j in hour t : y_{jt} equals 1 if unit j starts up at the beginning of hour t , and 0 otherwise.
z_{jt}	Shutdown status of unit j in hour t : z_{jt} equals 1 if unit j shuts down at the beginning of hour t , and 0 otherwise.
$p_{j t \omega}$	Power produced by unit j in hour t under scenario ω (MW).
$p_{i t \omega}^{\text{WS}}$	Power spillage of weather-dependent generating unit i in hour t under scenario ω (MW).
$D_{t \omega}^{\text{LS}}$	Unserved demand in hour t under scenario ω (MW).

Introduction

The unit commitment problem is widely used in the operation of power systems to minimize scheduling and operation costs. Increasing integration of renewable energy brings additional uncertainty into the power system, which challenges deterministic unit commitment formulations. A stochastic unit commitment (SUC) model allows representing the uncertainties from demand and weather-dependent generation units [1]. However, the practical application of a SUC model is limited due to its computational burden.

The SUC problem is typically formulated as a two-stage mixed-integer linear programming problem. The commitment decisions are represented in the first stage, while scenario-based operating conditions are modeled in the second stage. The binary commitment variables link both stages. In order to accurately represent the uncertainty, a sufficiently large number of scenarios is needed. However, with an increasing number of scenarios, the size of the SUC problem dramatically increases, which increases its computational burden, eventually leading to intractability.

The SUC problem has been studied for the past two decades [2, 3]. Diverse remedies to its computational burden (that more often than not leads to intractability) have been proposed in the literature [4–6].

Such remedies include decomposition, parallelization, and scenario reduction. Among them, Papavasiliou et al. [4] propose a parallel Lagrangian relaxation coupled with a scenario selection algorithm. Ryan et al. [5] propose a scenario clustering method to build stronger nonanticipative relaxation bounds that result in computational gains. Additionally, an integrality constraint relaxation method is proposed in [6], which reduces the computational burden.

In recent years, a number of learning-based methods have been proposed to address complex power system optimization problems, including optimal power flow, economic dispatch, and unit commitment. These learning-based methods can be divided into two categories [7]: (i) *end-to-end learning*, which aims to obtain near-optimal solutions by (just) learning [8–10]; and (ii) *learning-to-assist optimization*, which aims to reduce the solution time of the corresponding problem [11–18].

For the end-to-end learning, Chen et al. [8] propose a machine learning model to estimate quickly the optimal solution of the security-constrained economic dispatch problem. Park et al. [9] point out that the end-to-end learning approach suffers scalability issues due to a high-dimensional output space. They show that the optimal solution of an optimal power flow problem can be significantly compressed, which reduces the number of parameters that need to be trained. Moreover, Park et al. [10] use a graph neural network to

predict commitment decisions in unit commitment problems while identifying active bounds of transmission lines.

For the learning-to-assist optimization, Xavier et al. [11] use machine learning techniques to learn from historically solved security-constrained unit commitment instances to reduce the problem size and to provide an effective warm-start initial solution. A data-driven method proposed in [12] uses historical demand and renewable generation information to learn binding power flow limit constraints in the unit commitment problem. Hasan and Karagarian [13] present a method to convert system-state information into a color image and use a convolutional neural network to identify the redundant transmission constraints and binding ramping-up/down constraints. Jiménez-Cordero et al. [14] and Misra et al. [15] use learning methods to identify sets of binding constraints to reduce the sizes of the unit commitment and the optimal power flow problems, respectively. Baker [16] and Biagioni et al. [17] propose an iterative method with learning key primal and dual variables for quickly solving optimal power flow problems. Finally, Ramesh et al. [18] reduce the size of the unit commitment model by learning commitment decisions while introducing a subsequent feasibility checking layer. We note that existing learning-based methods require rather complicated learning strategies, and that no learning techniques for reducing the computational burden of the SUC problem have been found in the literature.

To reduce the computational burden of the SUC problem, we propose a simple learning method to reduce its size. Since the system operator solves the SUC problem daily, numerous historical input/output instances are available for learning. Learning from the output of historical solved instances with their corresponding weather-dependent generation and demand data (spanning, e.g., a month), we can identify the generators that show a stable on/off pattern and fix their binary commitment variables, which has a major impact on solution time. In addition, we can study the periods when generators do not reach the ramping up/down limits and then eliminate those ramping up/down constraints, which helps to further reduce the problem size.

The proposed learning method is significantly simpler than other methods reported in the literature, e.g., the one reported in [11].

For learning/predicting redundant inequality constraints, Xavier et al. [11] train a k-nearest neighbor classifier. Alternatively, we examine historically solved instances and focus on inequality constraints whose “ramping usage ratios” are smaller than a pre-specific threshold in all instances and eliminate them. Such a simple rule is quite effective for the stochastic unit commitment problem.

For fixing binary variables, Xavier et al. [11] train a support vector machine model, which requires a rather involved training. Alternatively, we simply examine binary variable patterns that do not change across instances, and fix them as

binary constants. Again, such a simple rule is quite effective for the stochastic unit commitment problem.

The remaining paper is organized as follows. Section 3 provides the formulation of the SUC model. Section 4 shows the detailed solution process, including the generation of proxies for historical instances and the simple learning procedure. The model reduction methods and solution quality metrics are presented as well. Section 5 provides the simulation results. Some conclusions are drawn in Section 6.

Model Assumptions and Formulation

In this section, we present an extensive description of the SUC problem, including assumptions and formulation.

Assumptions

1. The SUC problem is formulated as a two-stage stochastic programming problem.
2. As is customary, three commitment binary variables are used for each thermal unit.
3. Since this paper focuses on the impact of suppressing some ramping constraints and fixing some binary variables on computational efficiency and solution quality, we neglect transmission constraints.
4. Demand and weather-dependent generation (input data) are represented using a large enough number of scenarios.
5. Linear cost functions are used for the thermal generation units.
6. We assume that the outputs of nuclear generation units are known for the planning horizon.

Formulation

The formulation of the considered SUC is:

$$\begin{aligned} \text{Minimize}_{\Xi} \quad & \sum_t \left[\sum_j \left(c_{jt}^{\text{NL}} v_{jt} + c_{jt}^{\text{U}} y_{jt} + c_{jt}^{\text{D}} z_{jt} \right) \right. \\ & \left. + \sum_{\omega} \pi_{\omega} \left(\sum_j c_j^{\text{V}} p_{jt\omega} + \sum_i c_i^{\text{W}} p_{it\omega}^{\text{W}} + c^{\text{LS}} D_{t\omega}^{\text{LS}} \right) \right] \end{aligned} \quad (1a)$$

subject to:

$$v_{j,t-1} - v_{jt} + y_{jt} - z_{jt} = 0; \quad \forall j, \forall t \quad (1b)$$

$$v_{j,0} = v_j^{\text{ini}}; \quad \forall j \quad (1c)$$

$$\sum_{k=1}^{L_j} (1 - v_{jk}) = 0; \quad \forall j \quad (1d)$$

$$\sum_{k=t}^{t+T_j^{\text{U}}-1} v_{jk} \geq T_j^{\text{U}} y_{jt}; \quad \forall j, L_j < t \leq |T| - T_j^{\text{U}} + 1 \quad (1e)$$

$$\sum_{k=t}^{|T|} (v_{jk} - y_{jt}) \geq 0; \quad \forall j, t > |T| - T_j^U + 1 \quad (1f)$$

$$\sum_{t=1}^{F_j} v_{jt} = 0; \quad \forall j \quad (1g)$$

$$\sum_{k=t}^{t+T_j^D-1} (1 - v_{jk}) \geq T_j^D z_{jt}; \quad \forall j, F_j < t \leq |T| - T_j^D + 1 \quad (1h)$$

$$\sum_{k=t}^{|T|} (1 - v_{jk} - z_{jt}) \geq 0; \quad \forall j, t > |T| - T_j^D + 1 \quad (1i)$$

$$\sum_j p_{jt\omega} + \sum_i (p_{it\omega}^W - p_{it\omega}^{WS}) + \sum_{\ell} p_{\ell t}^N = D_{t\omega} - D_{t\omega}^{LS}; \quad \forall \omega, \forall t \quad (1j)$$

$$0 \leq p_{it\omega}^{WS} \leq p_{it\omega}^W; \quad \forall i, \forall \omega, \forall t \quad (1k)$$

$$0 \leq D_{t\omega}^{LS} \leq D_{t\omega}; \quad \forall \omega, \forall t \quad (1l)$$

$$\underline{P}_j v_{jt} \leq p_{jt\omega} \leq \overline{P}_j v_{jt}; \quad \forall j, \forall \omega, \forall t \quad (1m)$$

$$p_{jt\omega} - p_{j,t-1,\omega} \leq R_j^U v_{j,t-1} + S_j^U y_{jt}; \quad \forall j, \forall \omega, \forall t \quad (1n)$$

$$p_{j,t-1,\omega} - p_{jt\omega} \leq R_j^D v_{jt} + S_j^D z_{jt}; \quad \forall j, \forall \omega, \forall t \quad (1o)$$

$$p_{j,0,\omega} = p_j^{\text{ini}}; \quad \forall j, \forall \omega \quad (1p)$$

The objective function (1a) includes (i) first-stage costs: no-load, start-up, and shut-down, and (ii) second-stage costs: operating and unserved energy. Since the production cost of any nuclear unit is constant throughout the scheduling horizon, it is not included in the objective function.

Constraints (1b) ensure the start-up, shut-down and running logic of the generating units (with commitment variables). Constraints (1c) enforce the initial status of thermal units. Constraints (1d) - (1i) enforce the minimum up and down time of thermal units. Constraints (1j) ensure the power balance per hour and scenario. The weather-dependent generation spillage is bounded by constraints (1k). Constraints (1l) bound the unserved energy per scenario. Constraints (1m) bound the actual generation of each thermal generating unit. Maximum ramping-up limits of generating units are enforced through constraints (1n), whereas their maximum ramping-down limits are ensured by constraints (1o). Constraints (1p) enforce the initial power output of each thermal unit.

Algorithm 1 History instance generating process

```

1: input: (a) Demand and weather generation data from day 1 to the
   last day of the learning period ( $d^{\text{end}}$ ). (b) System initial condition
   for day 1 (system condition in hour 24 of day 0).
2: Set day counter  $d = 1$ .
3: for  $d = 1$  to  $d^{\text{end}}$  do
4:   Solve the  $n$ -scenario SUC of day  $d$ .
5:   Fix commitment decision from SUC and solve a randomly
   selected 1-scenario economic dispatch for day  $d$ .
6:   if  $d < d^{\text{end}}$  then
7:     Save economic dispatch results in hour 24 as the initial
     condition for day  $d + 1$ .
8:     Update day counter  $d \leftarrow d + 1$ .
9:   end if
10: end for

```

Solution Procedure

To reduce the size of the SUC problem, the proposed learning method relies on the availability of historically solved instances. In this section, we present a procedure to generate proxies of “historical instances”, when such information is not available, based on historical data of demand and renewable generation. Additionally, we present the proposed learning method and solution quality metrics.

Proxies for Historical Instances

To generate proxies of “historical instances” that are realistic, we use the process shown in Algorithm 1. Initially, we solve an n -scenario SUC problem starting from the first day of the learning period with the system’s initial conditions. Then, we solve the corresponding 1-scenario economic dispatch problem (shown in Appendix A) for a randomly selected scenario. After that, we update the system’s initial value based on the solution of the economic dispatch problem in hour 24, and solve the SUC problem of the next day. This loop continues until the end of the learning period, typically of one month.

Learning

From the available instances (historical or generated), we can learn unchanged patterns of binary variables, and inequality constraints that are inactive throughout the planning horizon. This is explained below.

Binary Variables

Binary variables correspond to commitment statuses and start-up and shut-down decisions of the generation units. We identify the binary variables that present a stable 0/1 pattern throughout the day in all learning instances, i.e., we identify the binary variables that correspond to the generating units

whose commitment status remains unchanged throughout the learning period. These variables are fixed to their corresponding optimal values, becoming binary constants.

We note that infeasibilities resulting from fixing binary variables are mostly due to logical and inter-temporal constraints. To avoid such infeasibilities, we use a large learning horizon (e.g., two months) and only freeze binary variables that present a fixed pattern across days throughout the learning horizon. This is effective for the stochastic unit commitment problem and allows significantly reducing the number of binary variables.

Inequality Constraints

Regarding inequality constraints, we eliminate ramping constraints based on the “ramping usage ratios” defined below. $R_{jt\omega}^{\text{UR}}$ and $R_{jt\omega}^{\text{DR}}$ correspond to the ramping usage ratio for ramping up and ramping down constraints, respectively:

$$R_{jt\omega}^{\text{UR}} = \frac{P_{jt\omega}^* - P_{j,t-1,\omega}^*}{R_j^{\text{U}} v_{j,t-1}^* + S_j^{\text{U}} y_{jt}^*} > \delta \quad (2a)$$

$$R_{jt\omega}^{\text{DR}} = \frac{P_{j,t-1,\omega}^* - P_{jt\omega}^*}{R_j^{\text{D}} v_{jt}^* + S_j^{\text{D}} z_{jt}^*} > \delta \quad (2b)$$

where v_{jt}^* , y_{jt}^* , z_{jt}^* , and $p_{jt\omega}^*$ denote the solution of the SUC problem.

A pre-specified threshold δ (between 0 to 1) is used to decide whether a ramping constraint is assumed to be inactive or not. If the ramping usage ratio of a ramping constraint for a unit is greater than δ at least in one historical instance, the ramping constraint for the unit at the considered period is kept in the problem (all scenarios). Otherwise, the ramping constraint is eliminated.

Parameter δ allows implementing a trade-off between solution time and solution quality. If δ is close to 0, we eliminate fewer ramping constraints, which results in higher solution quality but smaller computational time reduction; and vice versa.

Additionally, we note that failing to identify a reduced number of binding ramping constraints (false negatives) does not generally create an infeasibility issue. However, eliminating binding ramping constraints may lead to underestimating the system's total cost. By fixing selected binary

variables and eliminating selected ramping constraints, the reduced SUC problem takes comparatively less time to solve.

Quality Metrics

The solution quality of the reduced SUC problem is measured by comparing its objective function value (cost) and the cost obtained from the original SUC problem with the same input data. We solve three types of SUC problems for comparison, as shown in Table 1:

1. Problem 1 is the original SUC problem (1). The cost obtained from Problem 1 is the benchmark for comparison. c_1^{L} represents the cost obtained from solving Problem 1 with the data in the learning set. On the other hand, c_{i1}^{T} is the cost obtained from solving Problem 1 with out-of-sample data set i , where $i \in \{1, 2, \dots, n\}$.
2. Problem 2 (SUC-V) is the SUC problem with some commitment variables fixed from learning outcomes. Since SUC-V is more restricted than SUC, the cost obtained from Problem 2 should be greater than or equal to the cost obtained from Problem 1.
3. Problem 3 (SUC-CV) is the SUC problem with some commitment variables fixed and some ramping constraints ignored, based on learning outcomes. The cost obtained from Problem 3 can be higher or lower than the cost obtained from Problem 1.

From solving the three problems using the learning scenario set and the out-of-sample testing sets, the expected outcomes are:

$$\begin{cases} c_1^{\text{L}} \leq c_2^{\text{L}} \end{cases} \quad (3a)$$

$$\begin{cases} c_{i1}^{\text{T}} \leq c_{i2}^{\text{T}} \end{cases} \quad \forall i \in \{1, 2, \dots, n\} \quad (3b)$$

We note that if the cost differences in the comparisons are smaller than the solver convergence tolerance (0.01%), we consider them noise. In this case, the solution of the reduced model has similar quality as the original one, and the corresponding costs do not need to satisfy (3).

Table 1 Cost comparison matrix

SUC Problem	Learning scenario set	Testing scenario set i
Problem 1: No learning (benchmark SUC)	c_1^{L}	c_{i1}^{T}
Problem 2: With learning variables (SUC-V)	c_2^{L}	c_{i2}^{T}
Problem 3: With learning variables & constraints (SUC-CV)	c_3^{L}	c_{i3}^{T}

Results

In this section, we present the performance of the proposed reduction method using the Illinois 200-bus system [19]. The SUC and economic dispatch problems are solved using CPLEX [20] under JuMP [21] on a server with an Intel Xeon E5-2680 processor clocking at 2.50 GHz and 60 GB of RAM.

Data

The Illinois 200-bus system [19] is a synthetic power system including 42 thermal units (coal and natural gas, 67.3% of total installed capacity), 1 nuclear power plant (16.8% of total installed capacity), and 6 wind farms (15.9% of total installed capacity). In the simulations, we consider that all thermal units require commitment (binary) variables.

Regarding data, 1-year demand data is available from [19]. The wind data is obtained from the System Advisor Model (SAM) [22]. Using the 8 years (2007–2014) of wind generation data available in the SAM database, we generate additional scenarios as explained below. Figure 1 shows wind farm generation output ratio in one calendar day (first Monday in June) for the eight years of available data.

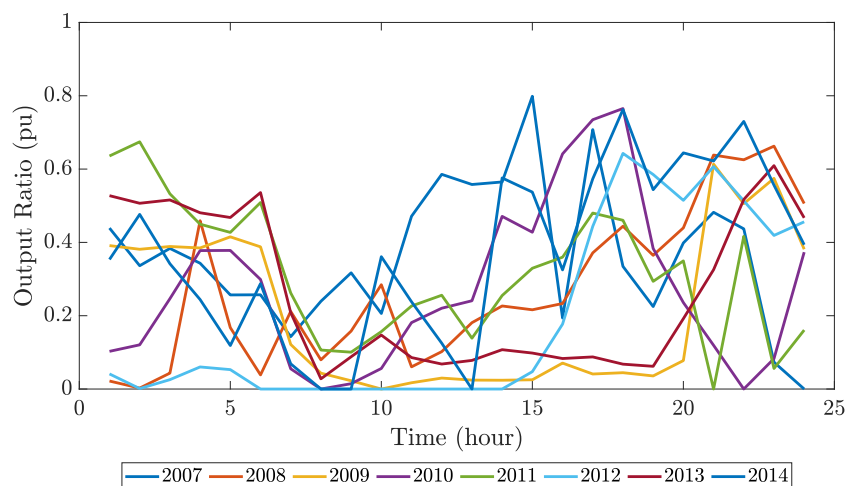
Scenarios

Demand

To expand the available demand data, we use daily convex combinations of 2-month (June and July) demand data in the data set to generate demand data for different scenarios. This is reasonable since the demand patterns for June and July are similar and can represent the typical demand in the summer season. The daily convex combination is:

$$D_{\omega e} = \alpha_{\omega e}^D D_e^A + (1 - \alpha_{\omega e}^D) D_e^B \quad \forall \omega, \forall e \quad (4)$$

Fig. 1 Wind farm generation output ratio in one calendar day (first Monday in June) for the eight years of available data



where $D_{\omega e}$ represents the generated demand data for day e of scenario ω . D_e^A and D_e^B correspond to historical demand data in months A and B for day e [19]. $\alpha_{\omega e}^D$ are random numbers between 0 and 1, generated using a Gaussian distribution.

Through this process, we obtain 100 scenarios of demand data each involving 1 month (4 weeks) of hourly values.

Wind Generation

We use hourly convex combinations of 2 of the 8 years of historical 1-month (June) wind data obtained from SAM [22] to generate different scenarios. The hourly convex combination is:

$$p_{i\omega t}^W = \beta_{\omega t}^W p_{it}^{W1} + (1 - \beta_{\omega t}^W) p_{it}^{W2} \quad \forall i, \forall \omega, \forall t \quad (5)$$

where $p_{i\omega t}^W$ represents wind power output of wind farm i in time period t of scenario ω . p_{it}^{W1} and p_{it}^{W2} correspond to 2 available wind generation scenarios for wind farm i in period t at bus n [22]. $\beta_{\omega t}^W$ are random numbers between 0 and 1, generated using a uniform distribution. Note that the wind farms in a specific region have similar wind output trends. Thus, the same random number is used for different wind farms during the same period in the same scenario.

Outcomes

Applying the scenario generation method to the available demand and wind generation data, we obtained 100 scenarios of data. We use the first 50 scenarios as the learning scenario set. The other 50 scenarios constitute the testing scenario set. For each scenario set, we include 28 days (instances) to represent each day in a month (4 weeks). The dimension of the input load data set is then $50 \times 24 \times 28$, which represents 50 scenarios of hourly load data in 28 days. The dimension of the input wind generation data set is $50 \times 6 \times 24 \times 28$,

Table 2 SUC Problem Size and Reduction Ratio ($\delta = 0$)

	v	y	z	Ramp Up	Ramp Down
Number	1,008	1,008	1,008	50,400	50,400
Fixing/Reduction Ratio	47.62%	69.05%	42.86%	62.30%	59.03%

representing 50 scenarios of 6 wind farms hourly generation data spanning 28 days.

Using Algorithm 1, we generate proxies for “historical instances” by inputting the learning scenarios. From the solved one-month instances, we identify the binary commitment variables that can be fixed, and the ramping constraints that can be eliminated.

The problem sizes and reduction ratios are shown in Table 2. Columns 1–3 correspond to the commitment variables (1008). The ratios of the binary variable v_{jt} , y_{jt} , and z_{jt} that can be fixed are 47.62%, 69.05%, and 42.86%, respectively. The learning outcome shows that the always-on units correspond to some large coal units with lower production costs. On the contrary, the always-off units correspond to some small natural-gas units. Columns 4 and 5 show that the number of ramping up/down constraints is 50400, which is proportional to the number of scenarios. We set the constraints suppression threshold δ to 0 to eliminate ramping constraints conservatively. The ratios of ramping-up and ramping-down constraints that can be eliminated are 62.30% and 59.03%, respectively, which mainly correspond to always-off units. Based on the learning outcome, we formulate two reduced SUC problems: SUC-V and SUC-CV.

For the learning test (in-sample analysis), we input the learning scenarios set to the reduced SUC problems and solve them from the first to the last day of the data set. We do not solve the economic dispatch problem to generate the initial condition in this test-solving procedure. Instead, we use the initial conditions saved from the learning process. Using the same initial condition for each day allows accurately comparing the solution quality for each instance. The solution performance and solution quality are illustrated in Table 3. The third column shows the average CPU time per instance in seconds. The fourth column provides the solution time saving as compared to the benchmark case. The fifth column shows the ratio of instances whose absolute relative cost difference (gap) with respect to the benchmark case is less than 0.01% (lower than the solver tolerance). The sixth column shows the average gap of all instances. Compared to the benchmark

case, SUC-V saves 89.60% of solution time. Meanwhile, all the instances result in identical solutions. Solving SUC-CV saves more than 90% of computing time and the solution quality is very high.

For the out-of-sample test, we input the testing scenarios set to the SUC problem to obtain the benchmark solution. Then, we use the reduced problems (SUC-V and SUC-CV), obtained from learning, to solve the same testing scenarios set. The solution outcome and solution quality are shown in Table 4. We note that the solution time-saving ratio for each reduced problem is similar to the one in the in-sample study. Even though the average gap for each reduced problem case is comparatively higher than that in the in-sample test outcome, the solution quality is very high in the out-of-sample results as well. The outcome shows that the reduced problems performed well in the out-of-sample data set and no infeasibility issue occurred during the study.

Conclusion

This paper presents a learning-based method to efficiently solve the SUC problem. Instead of using traditional decomposition techniques or scenario reduction methods to ease the computation burden inherent to the SUC problem, the proposed method uses historically solved instances to significantly reduce the size of the SUC problem. This is done by fixing binary variables that do not change and eliminating redundant ramping constraints. Compared to existing learning-to-assist methods, the proposed learning method is simple and does not require a sophisticated training process. From the computational experiments carried out, the conclusions below are in order:

1. The higher the number of binary variables frozen, the higher the computational efficiency. However, binary variables need to be fixed conservatively to avoid infeasibility. In reducing the SUC problems considered, we did not experience infeasibility issues as a result of using a long training horizon (one month).

Table 3 In-Sample Results (Learning Scenario Set)

No	Case	Avg. Time (s)	Time Saving	Gap<0.01%	Avg. Gap
1	SUC	638.0	-	-	-
2	SUC-V	66.4	89.60%	100%	0.000%
3	SUC-CV	41.7	93.46%	82.14%	0.009%

Table 4 Out-of-Sample Results (Testing Scenario Set)

No	Case	Avg. Time (s)	Time Saving	Gap<0.01%	Avg. Gap
1	SUC	608.6	-	-	-
2	SUC-V	64.7	89.37%	92.86%	0.002%
3	SUC-CV	42.6	92.99%	78.57%	0.010%

- The higher the number of ramping constraints suppressed, the higher the computational efficiency. However, eliminating ramping constraints needs to be done conservatively as well. This is so because such elimination may result in underestimating the total expected cost (scheduling and operation) and in committing fewer units than needed.
- The numerical results reported show that the reduced SUC problem produces high-quality solutions with highly reduced solution times.

Overall, the proposed learning-based method can be applied to practical applications since the method is straightforward, interpretable, and effective.

Declarations

Conflicts of interest The authors have no conflicts of interest to declare that are relevant to the content of this article. All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript. The authors have no financial or proprietary interests in any material discussed in this article.

Appendix A: Economic Dispatch

For learning and testing, the one-scenario economic dispatch problem is solved to update the system's initial status for next day SUC problem. The formulation of this problem is shown below:

$$\text{Minimize}_{p_{jt}, p_{it}^{\text{WS}}, D_t^{\text{LS}}} \sum_t \left(\sum_j c_j^{\text{V}} p_{jt} + \sum_i c_i^{\text{W}} p_{it}^{\text{W}} + c^{\text{LS}} D_t^{\text{LS}} \right) \quad (\text{A1a})$$

subject to:

$$\sum_j p_{jt} + \sum_i (p_{it}^{\text{W}} - p_{it}^{\text{WS}}) + \sum_{\ell} p_{\ell t}^{\text{N}} = D_t - D_t^{\text{LS}}; \quad \forall t \quad (\text{A1b})$$

$$0 \leq p_{it}^{\text{WS}} \leq p_{it}^{\text{W}}; \quad \forall i, \forall t \quad (\text{A1c})$$

$$0 \leq D_t^{\text{LS}} \leq D_t; \quad \forall t \quad (\text{A1d})$$

$$\underline{P}_j v_{jt}^* \leq p_{jt} \leq \overline{P}_j v_{jt}^*; \quad \forall j, \forall t \quad (\text{A1e})$$

$$p_{jt} - p_{j,t-1} \leq R_j^{\text{U}} v_{j,t-1}^* + S_j^{\text{U}} y_{jt}^*; \quad \forall j, \forall t \quad (\text{A1f})$$

$$p_{j,t-1} - p_{jt} \leq R_j^{\text{D}} v_{jt}^* + S_j^{\text{D}} z_{jt}^*; \quad \forall j, \forall t \quad (\text{A1g})$$

$$p_{j,0} = p_j^{\text{ini}}; \quad \forall j \quad (\text{A1h})$$

The objective function (A1a) is the operation cost. Constraints (A1b)–(A1h) are similar to constraints (1j)–(1p) but only considering a single scenario. In constraints (A1f) and (A1g), v_{jt}^* , y_{jt}^* , and z_{jt}^* are binary constants obtained from solving the SUC problem.

References

- Wu L, Shahidehpour M, Li T. Stochastic security-constrained unit commitment. *IEEE Transactions on Power Systems*. 2007;22(2):800–11.
- Conejo AJ, Carrión M, Morales JM. : Decision Making Under Uncertainty in Electricity Markets, vol. 1. New York: Springer; 2010.
- Roald LA, Pozo D, Papavasiliou A, Molzahn DK, Kazempour J, Conejo AJ. Power systems optimization under uncertainty: A review of methods and applications. *Electric Power Systems Research*. 2023;214: 108725.
- Papavasiliou A, Oren SS, Rountree B. Applying high performance computing to transmission-constrained stochastic unit commitment for renewable energy integration. *IEEE Transactions on Power Systems*. 2014;30(3):1109–20.
- Ryan K, Ahmed S, Dey SS, Rajan D, Musselman A, Watson J-P. Optimization-driven scenario grouping. *INFORMS Journal on Computing*. 2020;32(3):805–21.
- Ruiz PA, Philbrick CR, Sauer PW. Modeling approaches for computational cost reduction in stochastic unit commitment formulations. *IEEE Transactions on Power Systems*. 2009;25(1):588–9.
- Van Hentenryck, P.: Machine learning for optimal power flows. *Tutorials in Operations Research: Emerging Optimization Methods and Modeling Techniques with Applications*, 62–82 (2021)
- Chen W, Park S, Tanneau M, Van Hentenryck P. Learning optimization proxies for large-scale security-constrained economic dispatch. *Electric Power Systems Research*. 2022;213: 108566.
- Park, S., Chen, W., Mak, T.W., Van Hentenryck, P.: Compact optimization learning for AC optimal power flow. *arXiv preprint <http://arxiv.org/abs/2301.08840>* arXiv:2301.08840 (2023)
- Park, S., Chen, W., Han, D., Tanneau, M., Van Hentenryck, P.: Confidence-aware graph neural networks for learning reliability assessment commitments. *arXiv preprint <http://arxiv.org/abs/2211.15755>* arXiv:2211.15755 (2022)

11. Xavier, Á.S., Qiu, F., Ahmed, S.: Learning to solve large-scale security-constrained unit commitment problems. *INFORMS Journal on Computing* 33(2), 739–756 (2021). **This is a seminal and relevant paper regarding the application of learning techniques to reduce the size of the unit commitment problem.**
12. Pineda S, Morales JM, Jiménez-Cordero A. Data-driven screening of network constraints for unit commitment. *IEEE Transactions on Power Systems*. 2020;35(5):3695–705.
13. Hasan F, Kargarian A. Topology-aware learning assisted branch and ramp constraints screening for dynamic economic dispatch. *IEEE Transactions on Power Systems*. 2022;37(5):3495–505.
14. Jiménez-Cordero A, Morales JM, Pineda S. Warm-starting constraint generation for mixed-integer optimization: A machine learning approach. *Knowledge-Based Systems*. 2022;253: 109570.
15. Misra S, Roald L, Ng Y. Learning for constrained optimization: Identifying optimal active constraint sets. *INFORMS Journal on Computing*. 2022;34(1):463–80.
16. Baker K. Emulating AC OPF solvers with neural networks. *IEEE Transactions on Power Systems*. 2022;37(6):4950–3.
17. Biagioni, D., Graf, P., Zhang, X., Zamzam, A.S., Baker, K., King, J.: Learning-accelerated ADMM for distributed DC optimal power flow. *IEEE Control Systems Letters* 6, 1–6 (2020). **This article effectively exploits the learning-to-assist optimization idea for a multi-area optimal power flow problem solved via decomposition.**
18. Ramesh, A.V., Li, X.: Feasibility layer aided machine learning approach for day-ahead operations. arXiv preprint <http://arxiv.org/abs/2208.06742> arXiv:2208.06742 (2022)
19. Birchfield AB, Xu T, Gegner KM, Shetye KS, Overbye TJ. Grid structural characteristics as validation criteria for synthetic networks. *IEEE Transactions on Power Systems*. 2016;32(4):3258–65.
20. CPLEX V12.1: User's manual for CPLEX. International Business Machines Corporation, (2009)
21. Dunning I, Huchette J, Lubin M. Jump: A modeling language for mathematical optimization. *SIAM Review*. 2017;59(2):295–320.
22. Blair, N., Dobos, A.P., Freeman, J., Neises, T., Wagner, M., Ferguson, T., Gilman, P., Janzou, S.: System Advisor Model, SAM 2014.1.14: General Description. Technical Report, National Renewable Energy Lab.(NREL), Golden, CO (United States) (2014)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.